Práctica 1

Mariia Terekhova

10 de octubre de 2025

índice general

1. Ejercicio 1 **1**

1.1 Enunciado del Ejercicio 1

1.1.1 Estructura del Árbol de Procesos 4

1.1.2 Resultado Esperado 5

1.2 Identificación y Justificación de Tareas 6

1.2.1 Recepción y Validación de Argumentos 6

1.2.2 Generación de la Jerarquía de Procesos 6

1.2.3 Ejecución de Comandos Externos 6

1.2.4 Temporización sin Bloqueo 7

1.2.5 Comunicación y comportamiento Reactivo Mediante Señales 7

1.2.6 Destrucción Controlada de Árbol 7

1.3 Diagrama de Secuencia 10

1.4 Análisis Detallado del Código Fuente 11

1.4.1 Análisis del Apartado A 11

Función main() 11

Llamadas al Sistema Utilizadas: 11

Bloque de Generación del Árbol 11

Llamadas al Sistemas Utilizadas: 11

Bloque de Comprobación 12

Llamadas al Sistemas Utilizadas:. 12

1.4.2 Análisis del Apartado B. 13

Manejadores de Señales 13

Llamadas al Sistemas Utilizadas: 13

Función main() 13

Llamadas al Sistemas Utilizadas: 15

2. Ejercicio 2

2.1 Enunciado del Ejercicio 16
2.1.1 Resultado Esperado 16
2.2 Identificación y Justificación de Tareas 17
2.2.1 Recepción de Parámetros y Preparación del Archivo 17
2.2.2 Cálculo Dinámico de Hijos Necesarios 17
2.2.3 Configuración de la Comunicación Inter-Procesos 17
2.2.4 Rol del Procesos Padre: Lectura, Bombeo y Sincronización 18
2.2.5 Rol del Proceso Hijo: Creación de Archivos y Volcado de Datos 19
2.3 Diagrama de Secuencia 20
2.4 Análisis Detallado del Código Fuente
Bloque de Inicialización y Preparación del Archivo . . 21
Llamadas al Sistema Utilizadas 21
Bloque de Cálculo de Dimensiones 21
Llamadas al Sistema Utilizadas. 21
Bloque de Creación de Tuberías y Bifurcación 22
Llamadas al Sistema Utilizadas 22
Rama del Proceso Hijo: Recepción y Volcado de Datos 23
Llamadas al Sistema Utilizadas 23
Rama del Proceso Padre: Lectura, Envío y Sincronización 23
Llamadas al Sistemas Utilizadas 23

3. Ejercicio 3

3.1 Enunciado del Ejercicio 24
3.1.1 Estructura del Árbol de Procesos 24
3.1.2 Resultado Esperado 24
3.2 Identificación y Justificación de Tareas 25
3.2.1 Recepción de Parámetros y Dimensionamiento 25
3.2.2 Reserva y Vinculación de Memoria Compartida 25
3.2.3 Generación de la Rama Vertical y Registro de Ascendencia 25
3.2.4 Generación de los Nodos y Lectura de Ancestros 26
3.2.5 Sincronización Ascendente y Emisión del Superpadre 26
3.2.6 Desvinculación y Destrucción Controlada de Recursos 27
3.3 Diagrama de Secuencia 28
3.4 Análisis Detallado del Código Fuente 29
Bloque de Inicialización y Configuración de Memoria Compartida 29
Llamadas al Sistema Utilizadas 29
Bloque de Construcción de la Cadena Vertical 29
Llamadas al Sistema Utilizadas 29
Bloque de Generación de Nodos Hoja y Comportamiento del Subhijo 30
Llamadas al Sistema Utilizadas 30
Bloque de Sincronización Final y Limpieza del Sistema 31
Llamadas al Sistema Utilizadas 32

Capítulo 1
Ejercicio 1

1.1. Enunciado del Ejercicio

a) Realiza un programa llamado **malla.c** que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "pstree –c".

b) Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento.

Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.1.1. Estructura del Árbol de Procesos

La jerarquía de procesos que se debe crear del apartado **a** es la siguiente:

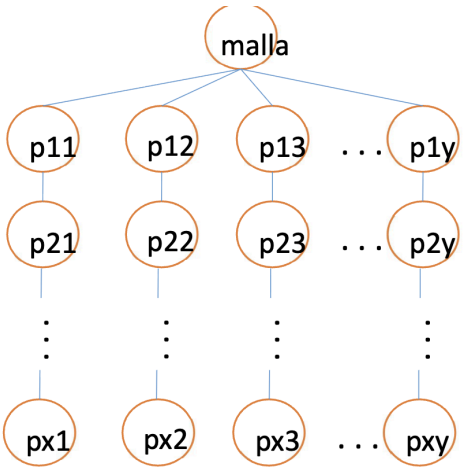


Figura 1.1: Diagrama del árbol de procesos a generar.

La jerarquía de procesos que se debe crear del apartado b es la siguiente:

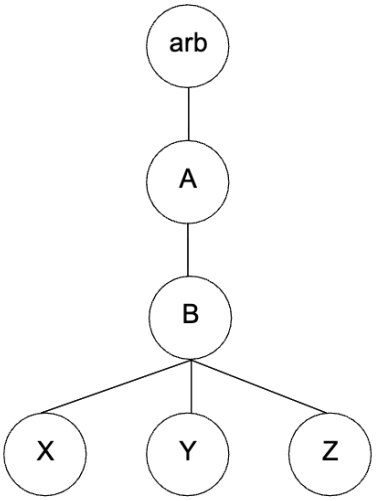


Figura 1.1: Diagrama del árbol de procesos a generar.

1.1.2. Resultado Esperado

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ ejec A 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
```

```
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752...

/* Tras un intervalo de 15 segundos aparecera */
Soy el proceso A con pid 752, he recibido la senyal.
/* Resultado del comando pstree */

Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

1.2. Identificación y Justificación de Tareas

1.2.1. Recepción y Validación de Argumentos

En el apartado A, se definen el ancho (y) y la profundidad (x) del árbol. En el apartado B, se establece el tiempo de espera del temporizador y, de forma conceptual, se describe el objetivo del proceso de la señal.

Se revisa la variable **argc** para asegurar que el usuario ingrese el número exacto de parámetros. Luego, se usa la función **atoi()** para transformar las cadenas de texto de **argv** en números que se puedan usar. Si la verificación no funciona, el programa se cierra de manera segura con **exit(1)**.

1.2.2. Generación de la Jerarquía de Procesos

El programa tiene que crear estructuras de árbol específicas. En el apartado A, se necesita una jerarquía dinámica que funcione con bucles. En el apartado B, se debe hacer un árbol fijo y desigual, como **ejec ↠ A ↠ B ↠ {X, Y, Z}**.

Se usa la función **fork()** del sistema de manera precisa. En el apartado A, bucles for anidados controlan cómo se expande el árbol en horizontal y en vertical. En el apartado B, se usa una serie de condiciones **if (pid==0)** para generar cada parte de la cadena de forma secuencial.

1.2.3. Ejecución de Comandos Externos

Se necesita ejecutar órdenes del sistema, como **pstree** o **ls**, para observar el estado del árbol o reaccionar a eventos, sin que el proceso principal que invoca el comando pierda su hilo de ejecución original o finalice prematuramente.

En el apartado A, se delega la ejecución en un proceso hijo efímero creado con un **fork()** adicional. Este hijo llama a la familia **exec**, como **execlp("pstree", ...)**, para reemplazar su imagen de memoria por la del comando, mientras el proceso padre original espera a que termine. En el apartado A, se utiliza **sprintf()** para pasar el PID raíz como argumento de texto al comando.

1.2.4. Temporización sin Bloqueo

En el apartado B, el proceso Z debe implementar una espera asíncrona durante un tiempo determinado por el argumento, sin usar la llamada al sistema **sleep**.

Para lograr esto, se usa la llamada al sistema **alarm()** para pedir al sistema operativo que envíe una señal **SIGALRM** después de un número de segundos especificado. Inmediatamente después, el proceso se suspende usando **pause()**, lo que le permite ceder la CPU hasta que la señal llegue y lo despierte. Esta forma de espera es eficiente porque no consume recursos mientras espera.

1.2.5. Comunicación y Comportamiento Reactivo Mediante Señales

En el apartado B, es necesario coordinar las acciones entre los procesos de la jerarquía. Las órdenes se envían desde el proceso Z hacia los procesos superiores como A, B, X, Y.

El proceso que envía usa la llamada **kill()** apuntando al PID del proceso destinatario. Envía señales como **SIGUSR1**. Los procesos que reciben tienen un comportamiento reactivo. Usan la función **signal()** para asociar la recepción de la señal a una rutina específica. Por ejemplo, **manejador_A o manejador_Z**.

1.2.6. Destrucción Controlada del Árbol

Existe una regla importante en la finalización de procesos: los padres nunca pueden terminar antes que sus hijos. Esto evita que se formen procesos huérfanos y asegura que el apagado ocurra de forma ordenada, en cascada.

En el apartado B, se requiere usar sistemáticamente la llamada **wait(NULL)** en todos los procesos que tengan descendencia. En ambos casos, los padres esperan hasta que **wait()** confirme que su hijo ha muerto. En el apartado B, esto produce una cadena de desbloqueos: X, Y y Z terminan con **exit(o)**, lo que libera los tres **wait()** del proceso B. Al morir B, libera al proceso A, permitiendo que todo termine en orden desde la raíz.

1.3. Diagrama de Secuencia

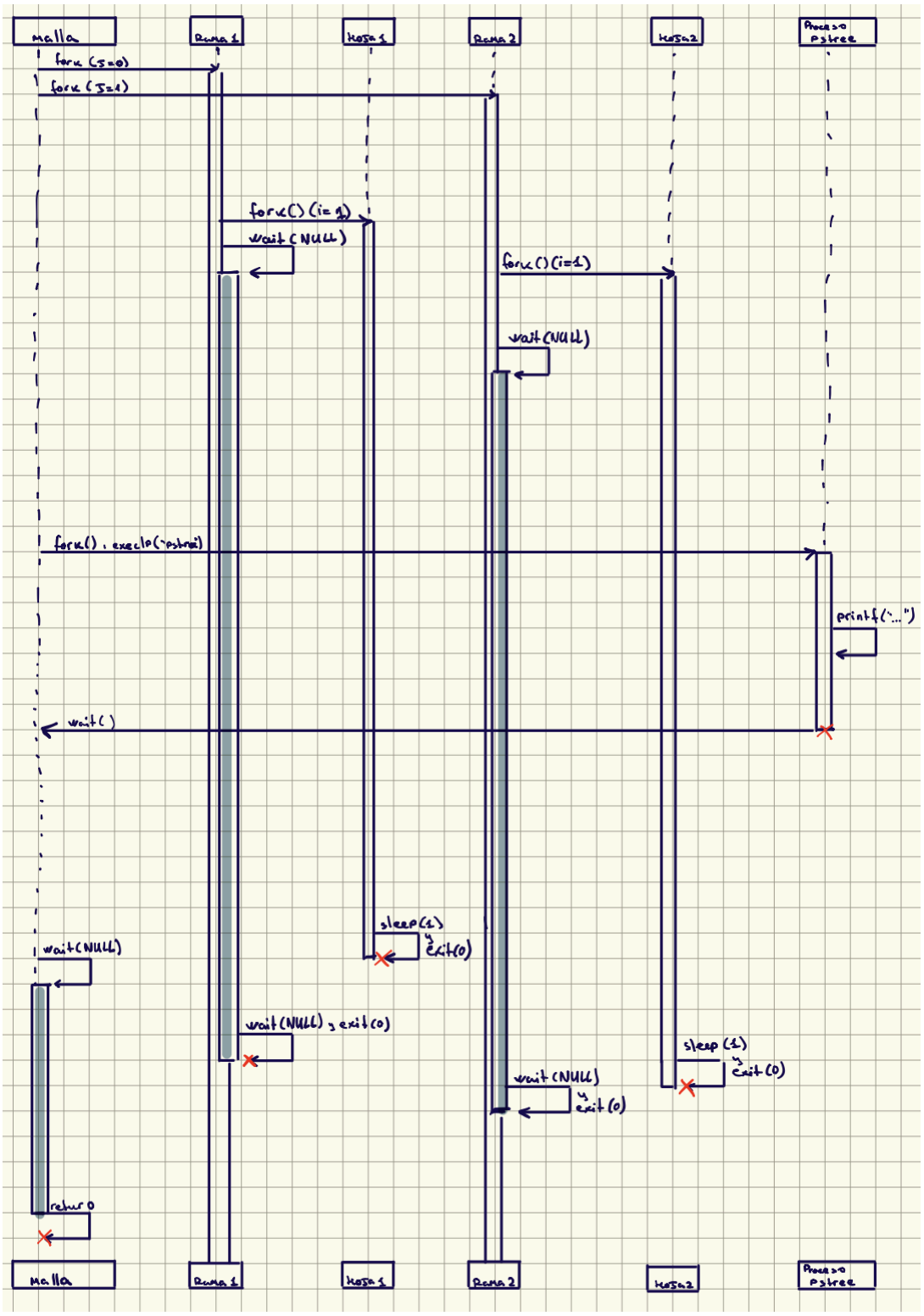


Figura 1.2: Diagrama de secuencia definitivo del apartado A con anidamiento estricto, etiquetas completas y cajas de activación partidas

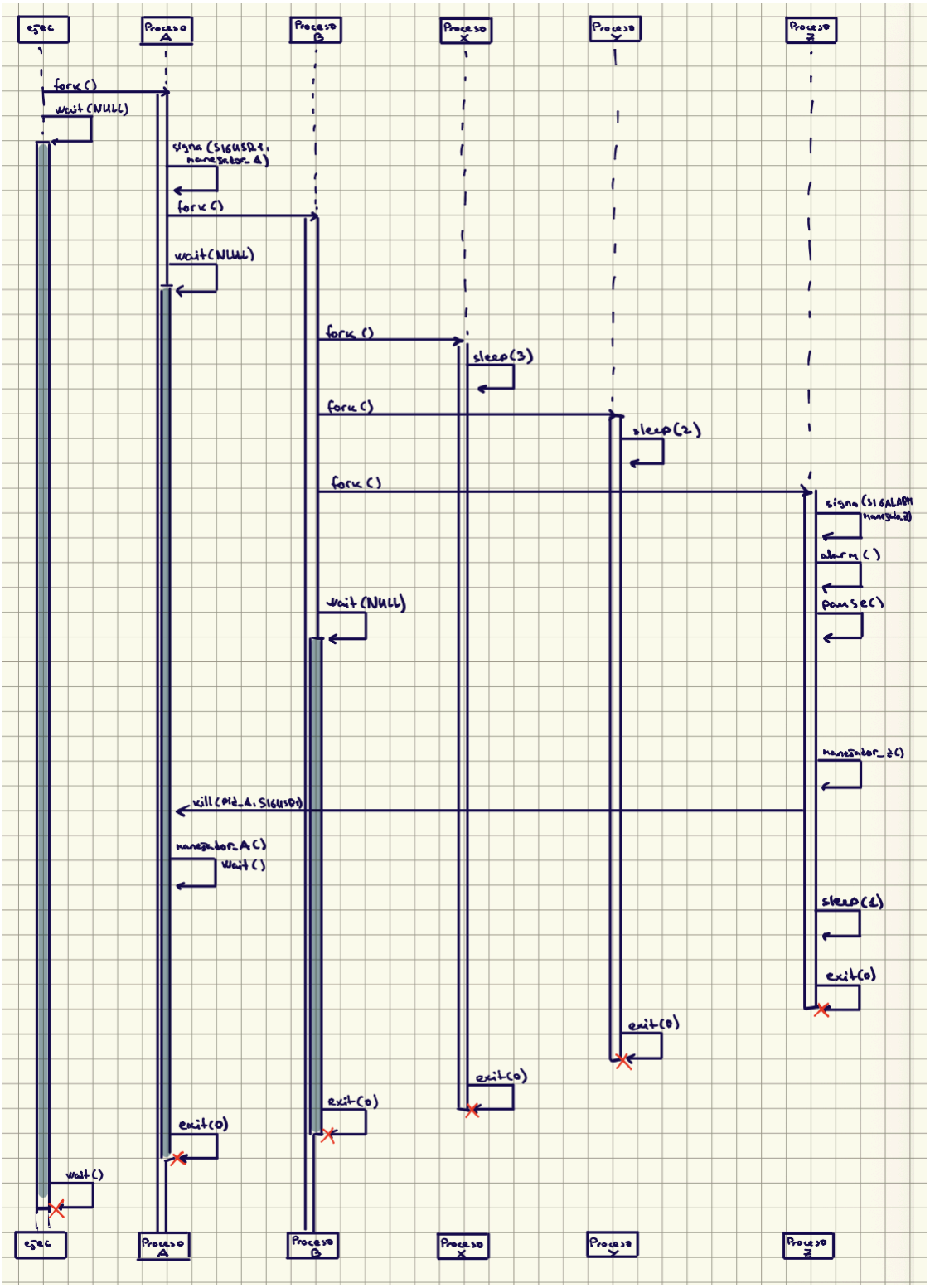


Figura 1.3: Diagrama de secuencia definitivo del apartado B con anidamiento estricto, etiquetas completas y cajas de activación partidas

1.4. Análisis Detallado del Código Fuente

1.4.1. Análisis del Apartado A

Función main():

El programa comienza en la función main, que es el punto de entrada. Aquí se reciben los argumentos de entrada llamados x e y. Estos valores se convierten a enteros usando la función **atoi()**. Si los argumentos no son válidos, el programa termina inmediatamente con un código de error. Luego, se guarda el identificador del proceso raíz con **getpid()**, porque más adelante se usará para ver la estructura del árbol.

Llamadas al Sistema Utilizadas: getpid(), exit()

```c
int main(int argc, char *argv[])
{
    if (argc != 3) {
        printf("Argumentos incorrectos\n");
        exit(1);
    }

    int x = atoi(argv[1]);
    int y = atoi(argv[2]);

    pid_t pid_raiz = getpid();
```

Bloque de Generación del Árbol

Dentro de main, se crea la lógica para construir el árbol de procesos. Un bucle for controlado por la variable y genera los hijos directos. Cada uno de estos hijos se crea con **fork()**. Luego, cada hijo entra en un segundo bucle, controlado por $x - 1$, que crea los subhijos, es decir, los procesos a mayor profundidad. Los subhijos esperan un poco y luego terminan. Los hijos directos, en cambio, hacen una pausa de un segundo con **sleep()** antes de terminar, para que el árbol se mantenga activo durante un tiempo.

Llamadas al Sistema Utilizadas: fork(), wait(), sleep(), exit(), perror()

```c
for (int j = 0; j < y; j++)
{
    pid_t pid = fork();
    if (pid == 0) {
        for (int i = 1; i < x; i++){
            pid_t child_pid = fork();
            if (child_pid > 0){
                wait(NULL);
                exit(0);
            }
            else if (child_pid < 0){
                perror("Error en fork");
                exit(1);
            }
        }
        sleep(1);
        exit(0);
    }
    else if (pid < 0) {
        perror("Error en fork");
        exit(1);
    }

}
```

Bloque de Comprobación (pstree) y Destrucción Controlada

Para ver cómo queda el árbol de procesos, el proceso raíz crea un último hijo auxiliar. Este hijo usa **execlp()** para ejecutar el comando pstree, pasándole como argumento el PID del proceso raíz que se guardó al principio. Mientras tanto, el proceso raíz entra en un bucle final donde llama a **wait(NULL)** tantas veces como hijos horizontales (*y*) haya creado. Esto asegura que el programa principal no termine hasta que todos los procesos hijos hayan terminado correctamente.

Llamadas al Sistema Utilizadas: fork(), execlp(), wait(), exit(), perror()

```c
    char pid_str[16];
    sprintf(pid_str, "%d", pid_raiz);

    if (fork() == 0) {

        execlp("pstree", "pstree", "-c", pid_str, NULL);
        perror("Error al ejecutar pstree");
        exit(1);
    }
    wait(NULL);
    for (int j = 0; j < y; j++) {
        wait(NULL);
    }
```

1.4.2. Análisis del Apartado B

Manejadores de Señales

Estas funciones no hacen nada dentro de su cuerpo, pero son fundamentales. Actúan como puntos de entrada cuando el sistema operativo envía una señal asíncrona al proceso. Su trabajo es evitar que el sistema tome acciones por defecto, como terminar el proceso, cuando recibe señales como **SIGALRM o SIGUSR1**. Al instalar los manejadores, se permite que llamadas como **pause()** se interrumpan de forma segura, lo que permite continuar con la ejecución normal del programa.

No se usan llamadas al sistema directamente dentro de estas funciones.

```
void manejador_A(int sig) {

}

void manejador_Z(int sig) {
}
```

Función main(): Generación del Tronco (ejec ↣ A ↣ B)

La función main es el punto de inicio del programa. Aquí se construye un árbol de procesos de forma secuencial. Cada vez que se ejecuta **fork()**, se crea un nuevo proceso hijo. Dependiendo del valor de pid, se decide si el proceso actual es el padre o el hijo. Cada proceso imprime su identificador con **getpid()**. El proceso A instala su manejador de señal para **SIGUSR1** en este momento. Luego, cada uno de los procesos del tronco — ejec, A y B — entra en una llamada a **wait()**, donde se detiene hasta que los procesos hijos terminan.

Llamadas al Sistema Utilizadas: getpid(), fork(), signal(), wait() y exit().

```
if (pid_A ==0) {
    pid_t num_pid_A=getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", num_pid_A, pid_ejec);
    signal(SIGUSR1, manejador_A);
    pid_t pid_B = fork();
```

```
if(pid_B==0) {
    pid_t num_pid_B=getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n", num_pid_B, num_pid_A, pid_e
    for(int i=0; i<3; i++){
        pid_t pid= fork();
        if(pid==0){
```

```
            wait(NULL); wait(NULL); wait(NULL);
            printf("Soy B(%d) y muero", num_pid_B);
            exit(0);

        }
        else if (pid_B>0){
            wait(NULL);
            printf("Soy A(%d) y muero",num_pid_A );
            exit(0);
        }
    }
```

Función main(): Bloque Lógico de los Procesos Hoja (X, Y, Z)

El proceso B crea tres procesos hijos: X, Y y Z. Para cada uno, se usa un bucle de tres iteraciones y se bifurca con una estructura **switch (i)** para definir su comportamiento.

Comportamiento pasivo (X, Y): Estos procesos no hacen mucho. Simulan trabajo con una espera temporal y después imprimen un mensaje indicando que han terminado.

Comportamiento activo y temporizador (Z): Este proceso es el más importante. Instala un manejador para capturar su propia señal de interrupción. Luego programa el reloj del sistema con **alarm()** y entra en un estado de espera eficiente, cediendo la CPU mediante la llamada **pause()**. Cuando el reloj suena, el sistema le envía la señal **SIGARLM**, lo que le despierta de la pausa y le permite continuar su ejecución para enviar la orden **SIGUSR1** al proceso A mediante la llamada **kill()**. Para su espera final antes de morir, Z vuelve a programar una breve alarma de un segundo y ejecuta un último pause().

Llamadas al Sistema Utilizadas: **getpid(), fork(), signal(), alarm(), pause(), kill() y exit().**

```
if(pid==0){
    switch (i){
        case 0:
            printf("Soy el proceso X: mi pid es %d.Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es
            alarm(tiempo+3);
            printf("Soy X(%d) y muero\n", getpid());
        break;
        case 1:
            printf("Soy el proceso Y: mi pid es %d.Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es
            alarm(tiempo+ 2);
            printf("Soy Y(%d) y muero\n", getpid());
        break;
        case 2:
            printf("Soy el proceso Z: mi pid es %d.Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es
            signal(SIGALRM, manejador_Z);
            alarm(tiempo);
            pause();

            kill(num_pid_A, SIGUSR1);

            alarm(1);
            printf("Soy Z (%d) y muero\n", getpid());
        break;
    }
    exit(0);
}
```

Capítulo 2
Ejercicio 2

2.1. Enunciado del Ejercicio

Realizar un programa llamado hacha.c que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, ... Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <**archivo**> <**tamaño**>

<**archivo**> nombre del archivo a dividir

<**tamaño**> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

2.1.1. Resultado Esperado

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

2.2. Identificación y Justificación de Tareas

2.2.1. Recepción de Parámetros y Preparación del Archivo

El programa debe funcionar desde la línea de comandos, donde el usuario indica el archivo que se va a procesar y el tamaño de cada fragmento. La sintaxis es: $ **hacha** <**archivo**> <**tamaño**>.

Se reciben dos argumentos: **argv[1]** para el nombre del archivo y **argv[2]** para el tamaño del fragmento, que se convierte a entero usando **atoi()**. Luego, se abre el archivo original con la función **open()**, utilizando la bandera **O_RDONLY** para asegurar que solo se lea. Esto devuelve un descriptor físico que permite acceder al archivo de forma segura. Todas las operaciones de lectura y escritura deben hacerse mediante llamadas al sistema.

2.2.2. Cálculo Dinámico de Hijos Necesarios

El enunciado exige que el proceso padre generará tantos procesos hijos como archivos se tengan que crear. Para saber este número exacto antes de empezar a crear procesos, es obligatorio calcular el tamaño total del archivo original.

Se utiliza la llamada al sistema **lseek()** con el parámetro **SEEK_END** para desplazar el cursor interno hasta el final del archivo y obtener su tamaño total en bytes. Luego se devuelve el cursor al principio con **SEEK_SET**. Después se divide el tamaño total entre el tamaño del trozo. Se emplea una sentencia condicional con el operador módulo (%) para añadir un hijo extra si la división no es exacta, y así cubrir el resto de bytes finales.

2.2.3. Configuración de la Comunicación Inter-Procesos

El requisito de que el proceso padre envíe información a los procesos hijos mediante tuberías exige usar un mecanismo de comunicación entre procesos, porque cada proceso tiene su propia memoria aislada.

Dentro de un bucle for, justo antes de llamar a **fork()**, el proceso padre crea una tubería con la llamada **pipe()**. De esta forma, cuando el proceso se clona, el hijo hereda los descriptores de archivo que apuntan a la misma tubería en memoria. Esto permite establecer un canal de comunicación efectivo entre el padre y cada hijo.

2.2.4. Rol del Proceso Padre: Lectura, Bombeo y Sincronización

El padre debe actuar como el distribuidor de la información, dividiendo el archivo original en partes y coordinando el envío a los hijos de forma secuencial o al mismo tiempo, para evitar que el sistema se sobrecargue.

En la rama de ejecución del padre (cuando **pid** > **0**), cierra primero su extremo de lectura del tubo con **close(fd_tuberia[0])**. Luego, usa un Array de Longitud Variable (VLA) con el tamaño exacto de cada trozo para leer esa cantidad de bytes del archivo original mediante **read()**, y envía todo ese contenido de una sola vez a través de la tubería con **write()**. Al cerrar su extremo de escritura, el padre envía una señal implícita de fin de archivo (EOF) al hijo. Finalmente, utiliza **wait(NULL)** para asegurar que los hijos se ejecuten uno a la vez, lo que garantiza que el padre no inicie al siguiente hijo hasta que el actual haya terminado su trabajo.

2.2.5. Rol del Proceso Hijo: Creación de Archivos y Volcado de Datos

El enunciado dice que los hijos serán los que creen los archivos de destino y escriban en ellos. Además, la generación de los nombres **(.h00, .h01)** debe cumplir las reglas de uso de funciones de la librería de C, sin usar **sprintf**.

En la rama del hijo **(pid == 0)**, el hijo cierra la salida de la tubería. Luego el hijo construye el nombre del archivo manualmente en memoria: el hijo copia el nombre base con **strcpy**, el hijo calcula la longitud con **strlen** y el hijo añade la extensión **.h** y los dígitos iterativos usando **aritmética ASCII** + **'0'.** Después de crear el archivo con **creat()**, el hijo entra en un bucle **while** donde el hijo usa **read()** para leer datos de la

tubería en bloques de 1024 bytes y el hijo escribe esos datos en su disco con **write()**. El hijo termina con **return 0** después de mover todos los datos.

2.3. Diagrama de Secuencia

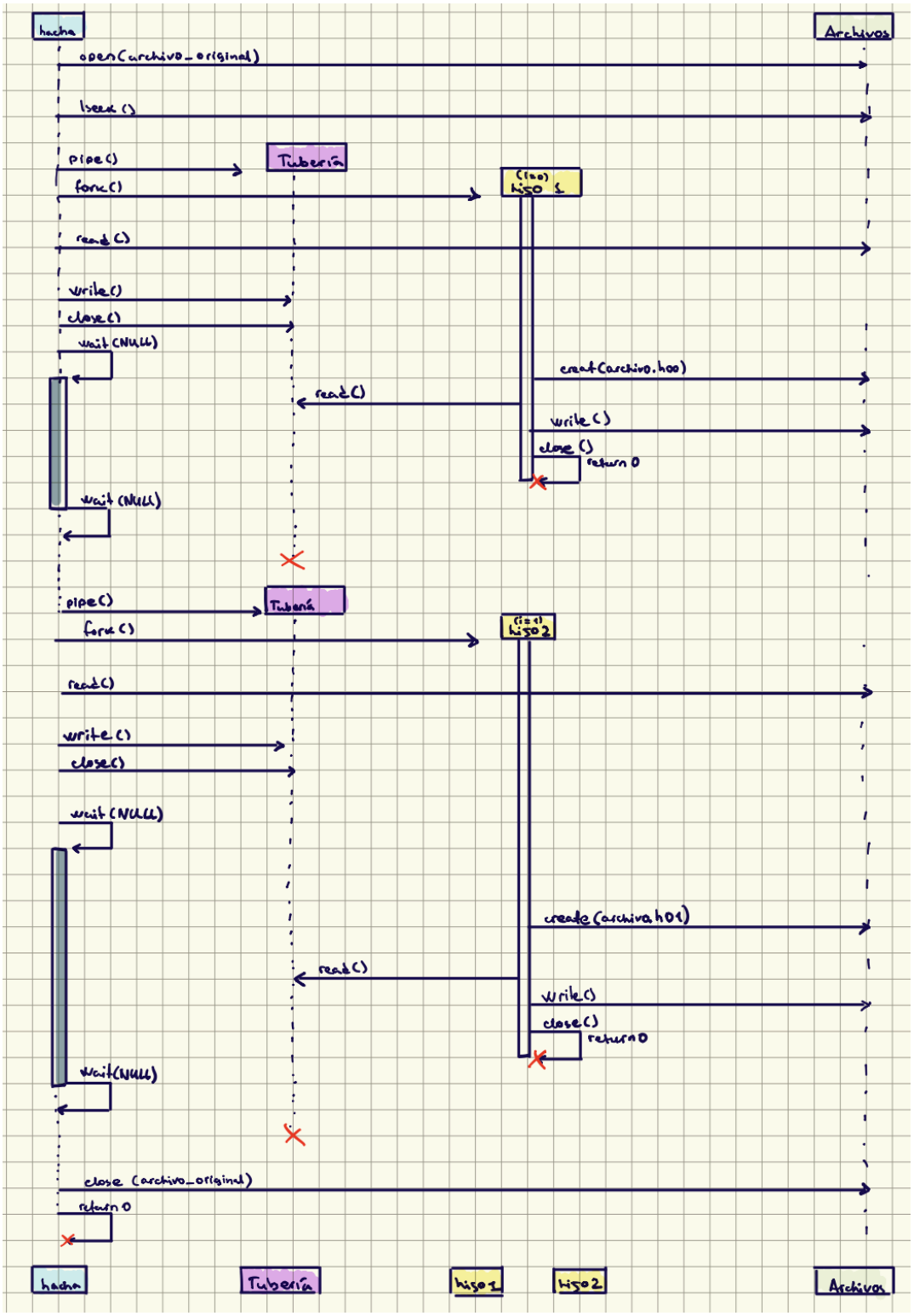


Figura 2.1: Diagrama de secuencia definitivo con anidamiento estricto, etiquetas completas y cajas de activación partidas.

Bloque de Inicialización y Preparación del Archivo

El programa comienza tomando los argumentos que el usuario pasa. El programa extrae el nombre del archivo que se va a dividir y convierte el tamaño del fragmento de texto a número entero usando **atoi()**. Luego el programa abre el archivo original en modo solo lectura **(O_RDONLY)**. Si el archivo no existe o no tiene permisos, el programa captura el error y termina la ejecución de forma segura.

Llamadas al Sistema Utilizadas: open(), perror()

```c
int main(int argc, char *argv[]) {

    char *nombre_archivo = argv[1];

    int tamano_trozo = atoi(argv[2]);

    int fd_original = open(nombre_archivo, O_RDONLY);
    if (fd_original < 0) {
        perror("Error al abrir el archivo original");
        return 1;
    }
```

Bloque de Cálculo de Dimensiones

Para cumplir con el requisito de generar tantos procesos hijos como archivos resultantes, el programa debe saber el tamaño total del archivo. Se mueve el cursor de lectura hasta el final **(SEEK_END)** del archivo para obtener la longitud en bytes. Luego se vuelve al inicio **(SEEK_SET)** del archivo para empezar a leer los datos. El número de procesos hijos se calcula dividiendo el tamaño total entre el tamaño de cada fragmento. Si hay algún byte que no entra en el reparto exacto, se agrega un proceso adicional.

Llamadas al Sistema Utilizadas: lseek()

```c
    long tamano_total = lseek(fd_original, 0, SEEK_END);
    lseek(fd_original, 0, SEEK_SET);
    int hijos = tamano_total / tamano_trozo;
    if (tamano_total % tamano_trozo != 0) {
        hijos++;
    }
```

Bloque de Creación de Tuberías

El núcleo del programa es un bucle **for** que se ejecuta tantas veces como hijos se hayan calculado. En cada paso del bucle, antes de clonar cualquier proceso, se pide al sistema operativo que cree un canal de comunicación unidireccional, como una **tubería**. Una vez que el canal está listo, se crea el nuevo proceso, dividiendo así la ejecución en dos partes: una para el proceso padre y otra para el proceso hijo.

Llamadas al Sistema Utilizadas: pipe(), perror(), fork()

```
for (int i = 0; i < hijos; i++) {
    int fd_tuberia[2];
    if (pipe(fd_tuberia) == -1) {
        perror("Error al crear la tubería");
        return 1;
    }

    pid_t pid = fork();
```

Rama del Proceso Hijo: Recepción y Volcado de Datos

La rama que corresponde al hijo (**pid** == **0**) actúa como receptor. Primero cierra el extremo de escritura de la **tubería**, porque solo va a leer datos. Luego trabaja directamente con cadenas en memoria, usando aritmética de punteros y códigos ASCII, para crear dinámicamente el nombre del archivo de destino. El nombre incluye un sufijo numérico como **.h00, .h01,** y así sucesivamente. Una vez determinado el nombre, solicita la creación del archivo en el disco. A continuación, entra en un bucle de lectura donde lee datos de la tubería en bloques de 1024 bytes. Cada bloque se escribe en el archivo correspondiente. Cuando la función read devuelve 0, significa que se ha llegado al final de la transmisión. En ese momento, el hijo cierra los descriptores de archivo y termina su ejecución.

Llamadas al Sistema Utilizadas: close(), creat(), perror(), read(), write()

```c
if (pid == 0) {

    close(fd_tuberia[1]);

    char nombre_destino[256];
    strcpy(nombre_destino, nombre_archivo);

    int len = strlen(nombre_destino);
    nombre_destino[len] = '.';
    nombre_destino[len + 1] = 'h';

    nombre_destino[len + 2] = (i / 10) + '0';
    nombre_destino[len + 3] = (i % 10) + '0';

    nombre_destino[len + 4] = '\0';

    int fd_destino = creat(nombre_destino, 0666);
    if (fd_destino < 0) {
        perror("Error al crear el archivo de destino");
        return 1;
    }

    char buffer[1024];
    int leidos;

    while ((leidos = read(fd_tuberia[0], buffer, sizeof(buffer))) > 0) {
        write(fd_destino, buffer, leidos);
    }

    close(fd_tuberia[0]);
    close(fd_destino);
    return 0;

}
```

Rama del Proceso Padre: Lectura, Envío y Sincronización

La rama principal (**pid** > **0**) actúa como emisor. Usa un vector que tiene el mismo tamaño que el fragmento que el usuario pidió. Con esto, extrae de una vez la parte correspondiente del archivo original. Si la lectura funciona bien, envía este bloque de datos al extremo de escritura de la tubería. Después de enviarlo, el padre cierra su conexión con la tubería. Esto envía una señal de fin de archivo que permite que el hijo lea. Por último, el padre ejecuta una espera bloqueante. Esto asegura que no empiece la siguiente iteración del bucle ni se cree un nuevo hijo hasta que el proceso actual termine de guardar su archivo en el disco.

Llamadas al Sistema Utilizadas: read(), write(), close(), wait()

```c
else  {
    char informacion[tamano_trozo];
    int leidos = read(fd_original, informacion, tamano_trozo);
    if (leidos > 0) {
        write(fd_tuberia[1], informacion, leidos);
    }
    close(fd_tuberia[1]);


    close(fd_tuberia[1]);

    wait(NULL);
}
```

Capítulo 3
Ejercicio 3

3.1. Enunciado del Ejercicio

Diseña un programa llamado hijos.c que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros.

3.1.1 Estructura del Árbol de Procesos

La jerarquía de procesos que se debe crear es la siguiente:

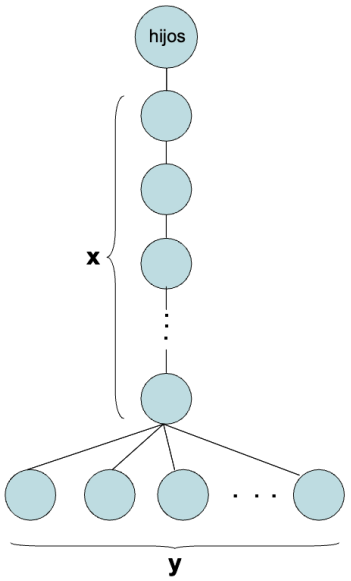


Figura 3.1: Diagrama del árbol de procesos a generar.

3.1.2 Resultado Esperado

A continuación se muestra un ejemplo de la salida por la terminal:

"Soy el superpadre (pid) : mis hijos finales son: `pidx1, pidx2, pidx3, …, pidxy`"

"Soy el subhijo pidx1, mi padres son: `pid1, pid2, …, pidx`"

3.2. Identificación y Justificación de Tareas

3.2.1. Recepción de Parámetros y Dimensionamiento

El programa necesita configurar la topología del árbol en tiempo real. Para ello, el programa recibe la profundidad de la rama vertical (x) y la anchura de las hojas finales (y).

El programa revisa y valida los argumentos de entrada, convirtiendo los argumentos de entrada a valores enteros con **atoi()**. Los valores de entrada definen la condición que detiene el bucle de creación de procesos y calculan el tamaño exacto (en bytes) de los segmentos de memoria con **sizeof(int)**.

3.2.2 Reserva y Vinculación de Memoria Compartida

Al usar **fork()**, los procesos hijos reciben copias independientes de la memoria del padre. Pero como los hijos finales necesitan saber los PIDs de sus padres, y el proceso raíz tiene que mostrar los **PIDs** de los hijos finales, es fundamental tener dos áreas de memoria RAM que todos los procesos de la jerarquía puedan acceder y modificar.

Para esto, se crean dos vectores. Primero, se reservan bloques físicos de memoria con **shmget()**, usando la clave **IPC_PRIVATE** para limitar el acceso solo a los procesos hijos, y se aplican los permisos **0666.** Luego, el proceso principal enlaza estos bloques a los punteros locales padres e hijos_finales con **shmat()**, así todos los hijos heredan el acceso directo a esas zonas desde el momento de su creación.

3.2.3 Generación de la Rama Vertical y Registro de Ascendencia

Se debe crear una cadena de procesos donde cada proceso padre genere exactamente un hijo (x veces). Cada proceso de esta cadena debe dejar registro de su identificador para que los procesos de la base puedan acceder al historial completo.

Con un ciclo for que se repita x veces, el proceso que está ejecutándose guarda su identificador (obtenido con **getpid()**) en la posición **padres[i]** de la memoria compartida. Luego ejecuta **fork()**. El proceso padre se queda esperando con

wait(NULL), mientras que el nuevo proceso hijo toma el control para seguir con la siguiente iteración del ciclo.

3.2.4 Generación de los Nodos Hoja y Lectura de Ancestros

El proceso en la parte inferior de la rama vertical debe crear y subhijos hermanos. Cada subhijo debe identificarse y mostrar la lista completa de sus ancestros.

Tras finalizar el bucle vertical, el proceso inferior hace un segundo bucle for con y iteraciones. Cada subhijo generado guarda su propio PID en el vector compartido **hijos_finales[j]**. Luego el proceso recorre el vector padres para imprimir la cadena de PIDs por pantalla aplicando un formato condicional **if-else** para gestionar las comas de separación, y finaliza su ejecución limpiamente.

3.2.5 Sincronización Ascendente y Emisión del Superpadre

El superpadre (el proceso inicial) necesita imprimir los PIDs de los subhijos finales. Esto crea una dependencia de tiempo estricta: el superpadre no puede leer el vector final hasta que los y subhijos hayan nacido y hayan escrito sus PIDs en el vector.

La sincronización se logra estructurando las sentencias **wait(NULL)** en anidamiento. Cuando los subhijos terminan, liberan a su creador; el creador, tras esperar a todos con otro bucle **wait()**, también termina. Esto produce un efecto dominó ascendente. El superpadre **(i == 0)** es el último proceso que despierta, momento en el que el vector **hijos_finales** está completamente lleno y listo para ser impreso en pantalla.

3.2.6 Desvinculación y Destrucción Controlada de Recursos

La memoria compartida es un recurso que existe en el núcleo del sistema operativo. Si los procesos no la gestionan correctamente, la memoria se queda ocupada de forma inadecuada. Esto se conoce como fuga de recursos.

Cada proceso del árbol ejecuta siempre la función **shmdt()** justo antes de terminar. Esto ocurre cuando el proceso devuelve 0. Además, de manera única y centralizada, el proceso principal, que es el superpadre **(i == 0)**, después de

imprimir su mensaje final y al saber que no hay otros procesos activos, llama a **shmctl()** con la opción **IPC_RMID**. Esta llamada elimina de forma definitiva los segmentos de memoria compartida del sistema.

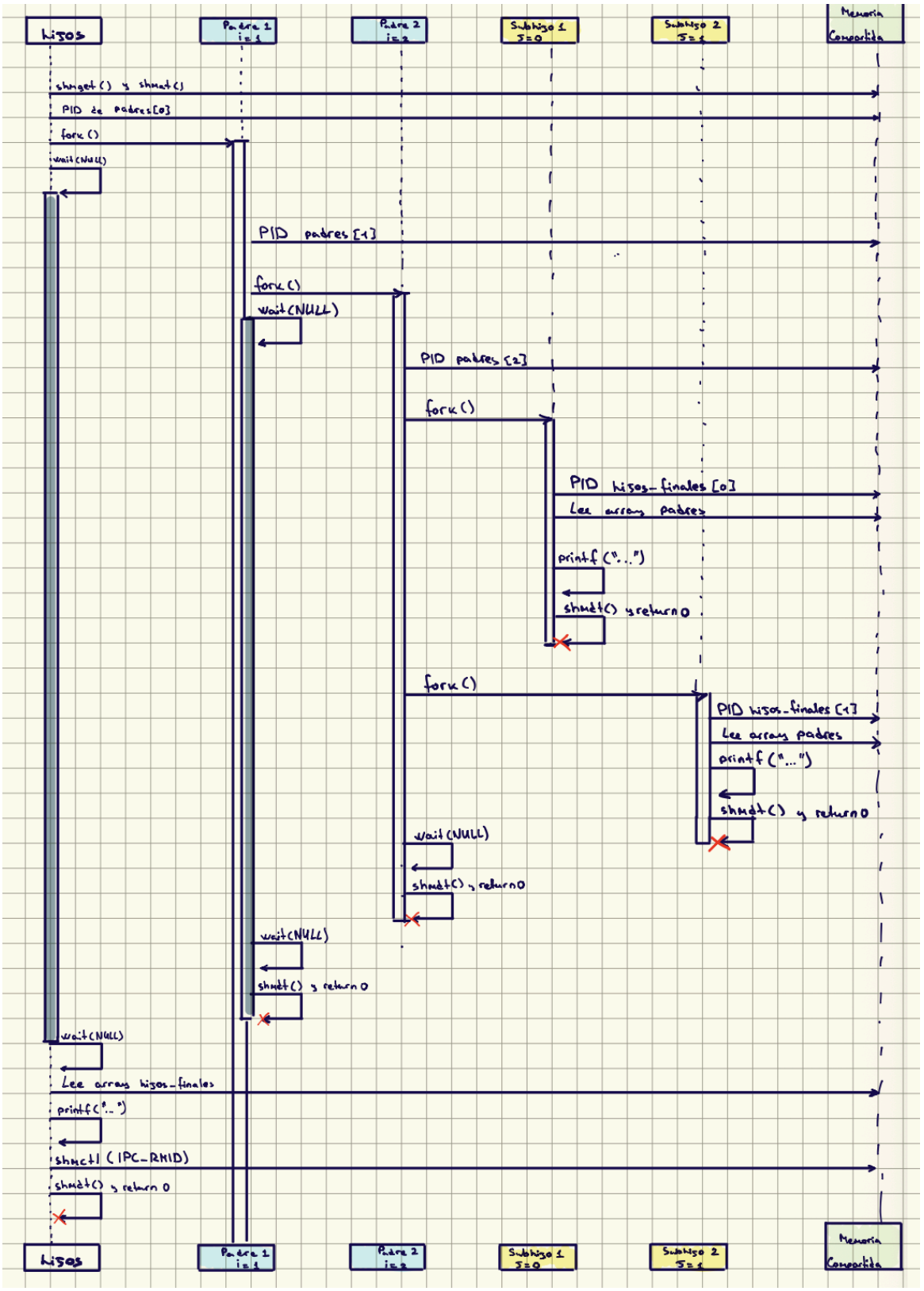


Figura 3.2: Diagrama de secuencia definitivo con anidamiento estricto, etiquetas completas y cajas de activación partidas.

3.4. Análisis Detallado del Código Fuente

Bloque de Inicialización y Configuración de Memoria Compartida

El programa empieza validando y convirtiendo los argumentos de entrada x e y a números enteros. Luego, configura los mecanismos de comunicación entre procesos. Solicita al sistema operativo crear dos segmentos de memoria compartida usando **IPC_PRIVATE**, lo que asegura que solo los procesos relacionados puedan acceder a ellos. Uno de los segmentos se reserva para almacenar x enteros, que son los PIDs de la rama vertical. El otro segmento se prepara para contener y enteros, correspondientes a los PIDs de los subhijos. Finalmente, estos segmentos físicos se conectan al espacio de direcciones del proceso principal mediante la asignación a los punteros **padres** e **hijos_finales.**

Llamadas al Sistema Utilizadas: shmget(), shmat()

```c
int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Uso: %s x y\n", argv[0]);
        return 1;
    }

    int x = atoi(argv[1]);
    int y = atoi(argv[2]);

    int shmid_padres = shmget(IPC_PRIVATE, x * sizeof(int), IPC_CREAT | 0666);
    int shmid_hijos  = shmget(IPC_PRIVATE, y * sizeof(int), IPC_CREAT | 0666);

    int *padres = (int *) shmat(shmid_padres, NULL, 0);
    int *hijos_finales = (int *) shmat(shmid_hijos, NULL, 0);
```

Bloque de Construcción de la Cadena Vertical

Este bloque crea un bucle **for** que construye la jerarquía de procesos en profundidad. En cada paso, el proceso actual guarda su identificador en el vector compartido **padres** con el índice **i**. Luego, crea una copia usando **fork()**. El proceso padre llama a **wait(NULL)** de forma bloqueante, deteniéndose hasta que su hijo termine. El proceso hijo, por su parte, ignora la llamada bloqueante y pasa a la siguiente iteración del bucle, convirtiéndose en el padre del siguiente nivel.

Llamadas al Sistema Utilizadas: getpid(), fork(), wait()

```c
for (int i = 0; i < x; i++) {

    padres[i] = getpid();

    if (i < x - 1) {
        pid_t pid = fork();

        if (pid > 0) {
            wait(NULL);

            if (i == 0) {
                printf("Soy el superpadre (%d) : mis hijos finales son: ", getpid());
                for(int j = 0; j < y; j++) {
                    if(j==y-1){
                        printf("%d", hijos_finales[j]);
                    }else{
                        printf("%d, ", hijos_finales[j]);
                    }

                }
                printf("\n");

                shmctl(shmid_padres, IPC_RMID, NULL);
                shmctl(shmid_hijos, IPC_RMID, NULL);
            }

            shmdt(padres);
            shmdt(hijos_finales);
            return 0;
        }
    }
}
```

Bloque de Generación de Nodos Hoja y Comportamiento del Subhijo

Cuando el bucle *x* termina, quedando solo el último padre activo en la cadena vertical, se empieza el bucle transversal que crea las hojas del árbol. En ese proceso principal se hacen *y* bifurcaciones. En la rama del hijo **(pid_hoja == 0)** el código guarda su propio PID en el vector compartido **hijos_finales**, lee el vector compartido padres paso a paso para imprimir toda su ascendencia con el formato requerido, desconecta sus punteros locales de la memoria compartida y termina su ejecución.

Llamadas al Sistema Utilizadas: getpid(), fork(), shmdt()

```c
for (int j = 0; j < y; j++) {
    pid_t pid_hoja = fork();
    hijos_finales[j] = getpid();
    if (pid_hoja == 0) {
        printf("Soy el subhijo (%d), mi padres son: ", getpid());
        for(int k = 0; k < x; k++) {
            if(k==x-1){
                printf("%d", padres[k]);
            }else{
                printf("%d, ", padres[k]);
            }
        }
        printf("\n");
        shmdt(padres);
        shmdt(hijos_finales);
        return 0;
    }
}
```

Bloque de Sincronización Final y Limpieza del Sistema

El último padre de la cadena espera a sus *y* subhijos con un bucle de **wait(NULL)**. Cuando esos procesos mueren, el padre se desbloquea, libera su memoria y luego muere, repitiendo este proceso hacia arriba.

Llamadas al Sistema Utilizadas: wait(), shmctl(), shmdt()

```c
for (int j = 0; j < y; j++) {
    wait(NULL);
}
shmdt(padres);
shmdt(hijos_finales);
return 0;
```

```
for (int j = 0; j < y; j++) {
    pid_t pid_hoja = fork();
    hijos_finales[j] = getpid();
    if (pid_hoja == 0) {
        printf("Soy el subhijo (%d), mi padres son: ", getpid());
        for(int k = 0; k < x; k++) {
            if(k==x-1){
                printf("%d", padres[k]);
            }else{
                printf("%d, ", padres[k]);
            }
        }
        printf("\n");
        shmdt(padres);
        shmdt(hijos_finales);
        return 0;
    }
}
for (int j = 0; j < y; j++) {
    wait(NULL);
}
shmdt(padres);
shmdt(hijos_finales);
return 0;
```